

■ COMPLETE NOTES - TOPIC 12

CSS Variables Custom Properties

Reusable values, themes, design tokens aur maintainable CSS - Basic se
Advanced tak

Web Development Course - Shauray

custom properties

var()

:root

scope

fallback

inheritance

themes

design tokens

calc()

media queries

CSS Variables Kya Hote Hain?

CSS values ko reusable naam dene ka modern method

SIMPLE DEFINITION

CSS variables, jinhe custom properties bhi bolte hain, reusable CSS values store karne ka tarika hain. Variable define karne ke liye name `--` se start hota hai, aur use karne ke liye `var()` function use hota hai.

Agar project mein same color, spacing, font size, shadow ya border radius multiple jagah use ho raha hai, to CSS variable se us value ko ek naam de sakte hain. Baad mein value change karni ho to sirf ek jagah update karna hota hai. Isse CSS clean, reusable aur theme-friendly ban jati hai.

● ● ● basic variable syntax

```
:root {
  --brand-color: #00b4d8;
  --card-radius: 8px;
}

.button {
  background: var(--brand-color);
  border-radius: var(--card-radius);
}
```

■ Why Variables Useful Hain?

Benefit	Explanation
Reusability	Same value ko baar-baar hardcode nahi karna padta
Maintainability	Color/spacing change ek central place se ho sakta hai
Theming	Light/dark theme ya brand themes easily switch ho sakte hain
Consistency	Design system mein same colors, gaps aur radii maintain rehte hain
Dynamic CSS	Variables media queries, hover states aur JS ke saath update ho sakte hain

■ **Core Idea:** CSS variable value store nahi karta sirf like a programming variable; it behaves as a CSS custom property with cascade, inheritance and scope.

Variables Define Kaise Karte Hain?

Custom property name hamesha double hyphen se start hota hai

DEFINITION RULE

CSS variable ka name `--` se start hota hai. Example: `--primary`, `--space-md`, `--font-body`.

Variable kisi bhi selector ke andar define ho sakta hai.

define variables

```
:root {  
  --primary: #2563eb;  
  --text-main: #111827;  
  --space-sm: 8px;  
  --space-md: 16px;  
  --radius: 10px;  
}
```

■ Naming Best Practices

Name Type	Example	Use
Color tokens	--color-primary	Brand and UI colors
Spacing tokens	--space-4	Padding, margin, gap
Font tokens	--font-heading	Typography system
Size tokens	--sidebar-width	Layout dimensions
Component tokens	--button-bg	Specific component values

■ **Naming Tip:** Variable names meaningful rakho. `--blue` ke bajay `--color-primary` zyada flexible hai, kyunki future mein primary color blue se green ho sakta hai.

var() Function

Defined variable ko actual CSS property mein use karna

`var()` function variable ki value read karta hai. Is function ke andar variable name pass karte hain.

Agar variable available hai, browser uski value use karega.

using var()

```
.card {  
  background: var(--surface);  
  color: var(--text);  
  padding: var(--space-md);  
  border-radius: var(--radius);  
}
```

VARIABLE DRIVEN CARD DEMO

--demo-accent controls left border color.

--demo-radius controls corner radius.

--demo-gap controls spacing between cards.

■ Where var() Can Be Used?

CSS Property	Example
color	color: var(--text-main)
background	background: var(--surface)
padding / margin	padding: var(--space-md)
border	border: 1px solid var(--border)
box-shadow	box-shadow: var(--shadow-lg)

■ **Syntax Warning:** Variable use karte time `var(--name)` likhna zaruri hai. Direct `color: --primary` invalid CSS hai.

:root, Scope and Inheritance

Variable kahan define hai, wahi decide karta hai kahan available hogi

IMPORTANT CONCEPT

:root document ka top-level selector hota hai. Isme define variables globally available hoti hain. Agar variable kisi component ke andar define ho, to woh us component aur uske children ke liye available hoti hai.

● ● ● global and local scope

```
:root {  
  --brand: #00b4d8; /* global */  
}  
  
.alert {  
  --brand: #e94560; /* local override */  
  border-color: var(--brand);  
}  
  
.alert strong {  
  color: var(--brand); /* inherited local value */  
}
```

■ Scope Rules

Place	Availability
:root	Usually entire document mein available
body	Body aur uske children mein available
.card	Sirf card aur card ke children mein available
.card:hover	Hover state mein variable override ho sakti hai

■ **Cascade Rule:** CSS variables cascade follow karti hain. Same name ki variable closer selector par define hai to local value global value ko override kar sakti hai.

Fallback Values in var()

Variable missing ho to backup value use karna

DEFINITION

`var()` function second argument as fallback accept karta hai. Agar variable defined nahi hai ya invalid hai, browser fallback value use karega.

● ● ● fallback syntax

```
.button {  
  background: var(--button-bg, #2563eb);  
  color: var(--button-text, white);  
}
```

■ Nested Fallback

● ● ● fallback chain

```
.card {  
  background: var(--card-bg, var(--surface, #ffffff));  
}
```

Code

Meaning

`var(--color)`

Variable exist kare to use karo, warna declaration invalid ho sakti hai

`var(--color, red)`

Variable missing ho to red use karo

`var(--a, var(--b, blue))`

--a missing ho to --b, --b missing ho to blue

■ **Practical Tip:** Reusable components banate time fallback values helpful hoti hain, kyunki component parent theme ke bina bhi usable rehta hai.

Design Tokens with Variables

Colors, spacing, typography aur shadows ko system banana

Design tokens woh named values hain jo design system ko consistent banate hain. CSS variables design tokens implement karne ka simple and powerful method hain. Example: primary color, page background, spacing scale, font sizes, border radius, shadows.

token system

```
:root {  
  /* colors */  
  --color-bg: #0f172a;  
  --color-surface: #1e293b;  
  --color-primary: #38bdf8;  
  
  /* spacing */  
  --space-1: 4px;  
  --space-2: 8px;  
  --space-4: 16px;  
  
  /* shape */  
  --radius-sm: 4px;  
  --radius-md: 8px;  
}
```

TOKEN FAMILY DEMO

--color-primary

--color-success

--color-danger

■ Token Categories

Category	Examples
Color	--color-bg, --color-text, --color-primary, --color-border
Spacing	--space-xs, --space-sm, --space-md, --space-lg
Typography	--font-body, --font-heading, --text-sm, --text-lg
Shape	--radius-sm, --radius-md, --radius-full
Effects	--shadow-sm, --shadow-lg, --transition-fast

Theme Switching with Variables

Same components, different variable values

THEME CONCEPT

Theme ka simple idea hai: component CSS same rahe, bas variables ki values change ho jayein. Isse light theme, dark theme, brand theme, success theme, danger theme easily create ho sakte hain.

● ● ● light and dark theme

```
:root {
  --bg: #ffffff;
  --text: #111827;
  --surface: #f3f4f6;
}

[data-theme="dark"] {
  --bg: #0f172a;
  --text: #e5e7eb;
  --surface: #1e293b;
}

body {
  background: var(--bg);
  color: var(--text);
}
```

■ Component Theme Override

```
.button {  
  background: var(--button-bg);  
  color: var(--button-text);  
}  
  
.button.danger {  
  --button-bg: #e11d48;  
  --button-text: #ffffff;  
}
```

■ **Professional Use:** Theme switching mein CSS variables bahut powerful hain, kyunki JavaScript ko har element style change nahi karna padta. Sirf root attribute/class change karna kaafi hota hai.

Variables with calc() and Media Queries

Reusable values ko calculation aur responsive design mein use karna

CSS variables numbers, sizes, colors, fonts sab store kar sakti hain. Jab variable length/number value hold karti hai, to usko `calc()` ke saath combine karke dynamic size create kar sakte hain.

● ● ● calc with variables

```
:root {
  --sidebar: 280px;
  --gap: 24px;
}

.main {
  width: calc(100% - var(--sidebar) - var(--gap));
}
```

■ Responsive Variable Override

● ● ● media query variables

```
:root {
  --container-padding: 32px;
  --section-gap: 80px;
}

@media (max-width: 700px) {
  :root {
    --container-padding: 18px;
    --section-gap: 48px;
  }
}
```

■ **Responsive Trick:** Component CSS same rakho, media query mein variables update karo. Isse repeated CSS kam hoti hai aur design consistent rehta hai.

CSS Variables and JavaScript

Runtime par CSS values change karna

DYNAMIC STYLING

CSS variables runtime par JavaScript se read aur update ho sakti hain. Ye theme toggle, color picker, progress bar, range slider, interactive dashboards aur user settings ke liye useful hai.

● ● ● set variable with javascript

```
document.documentElement.style.setProperty(  
  '--primary',  
  '#06d6a0'  
);
```

● ● ● read variable value

```
const styles = getComputedStyle(document.documentElement);  
const primary = styles.getPropertyValue('--primary');
```

■ JavaScript Kab Use Karein?

Situation	Example
User chooses theme color	Color picker se --primary update
Progress value changes	--progress: 65%
Slider controls spacing	--gap value update
App theme toggle	data-theme attribute change

■ **Performance Habit:** Agar many elements ka same visual value change karna hai, root variable update karna individual inline styles update karne se cleaner hota hai.

Common Mistakes and Fixes

CSS variables use karte waqt beginners ki common problems

MISTAKE

Variable name mein double hyphen bhoolna.
var() ke bina variable use karna.
Variable wrong scope mein define karna.
Color names ke hisaab se tokens banana.
Fallback value na dena reusable components mein.
Too many random one-use variables banana.

FIX

Always --name syntax use karo.
Use value as var(--name).
Global tokens :root mein define karo.
Purpose-based names use karo, jaise --color-primary.
Components mein fallback add karo.
Variables meaningful reusable values ke liye banao.

■ Debugging Checklist

Check	Question
Name	Kya variable <code>--</code> se start ho rahi hai?
Usage	Kya property mein <code>var(--name)</code> use kiya hai?
Scope	Kya variable element ke liye available hai?
Spelling	Definition aur usage mein exact same variable name hai?
Fallback	Variable missing ho to fallback required hai?
Valid value	Kya variable ki value property ke liye valid CSS value hai?

■ **Big Rule:** CSS variables cascade aur inheritance follow karti hain. Agar value expected nahi aa rahi, selector scope aur override order check karo.

CSS Variables - Complete Summary

Revision ke liye all important variable concepts ek jagah

Concept	Meaning	Example
Custom property	CSS variable definition	--primary: #00b4d8
var()	Variable value use karna	color: var(--primary)
:root	Global variables define karna	:root { --space: 16px; }
Fallback	Missing variable ke liye backup	var(--bg, white)
Scope	Variable kis element tree mein available hai	.card { --accent: red; }
Override	Same variable ka local value set karna	.dark { --bg: black; }
Design tokens	Named design system values	--color-primary, --space-md
Theme	Variable values switch karke UI change	[data-theme="dark"]

■ Fast Decision Guide

Need	Choose
Same color multiple places use ho raha hai	--color-primary variable
Consistent spacing system chahiye	--space-xs/sm/md/lg tokens
Light/dark theme banana hai	:root + [data-theme] variable overrides
Reusable component banana hai	Component variables + fallback values
Responsive spacing change karni hai	Media query mein variable override
JS se style value change karni hai	setProperty('--name', value)

■ Best Practices - Yaad Rakhna

DO

Global design tokens :root mein define karo.
Purpose-based variable names use karo.
var() ke saath fallback use karo where needed.
Theme values variables se switch karo.
Repeated colors, spacing, radii ko variables
banao.
Scope aur cascade samajh kar override karo.

AVOID

Random hardcoded values everywhere.
One-use unnecessary variables.
Color-specific names that limit future changes.
Variables wrong component scope mein define
karna.
Missing fallback in reusable widgets.
Spelling mismatch between define and use.

> NEXT TOPICS TO COVER

[Flexbox](#)[CSS Grid](#)[Media Queries](#)[Responsive Design](#)[Transitions](#)[Animations](#)[Forms Styling](#)[Pseudo Classes](#)

Made with ❤️ for Web Development Course learners

Shauray ke saath CSS seekhte raho 🚀 | Topic 12 - CSS Variables